Memory Efficient Doubly Linked List – Delete At Begining –Program Analysis and Time Complexity

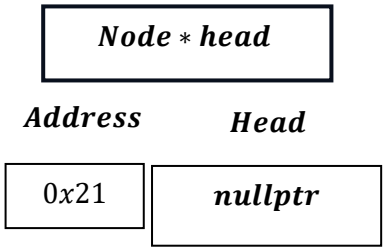
Program:

```
void deleteFromBeginning()
{
    if (head == nullptr)
    {
        cout << "List is empty. Nothing to delete." << endl;
        return;
    }

    Node *temp = head;
    // Decode the second node (which will become head)
    // Next = NULL ^ head->npx
    Node *next = XOR(nullptr, head->npx);

    if (next != nullptr)
    {
        // Update the new head's npx.
        // Currently next->npx is: XOR(head, nextNext)
        // We need it to be: XOR(NULL, nextNext)
        Node *nextNext = XOR(temp, next->npx);
        next->npx = XOR(nullptr, nextNext);
    }

    head = next;
    cout << "Deleted " << temp->data << " from the
beginning." << endl;
    free(temp);
}
```

Program – Analysis

1. *When List Is Empty*

```
    if (head == nullptr)
    {
        cout << "List is empty. Nothing to delete." << endl;
        return;
    }
```

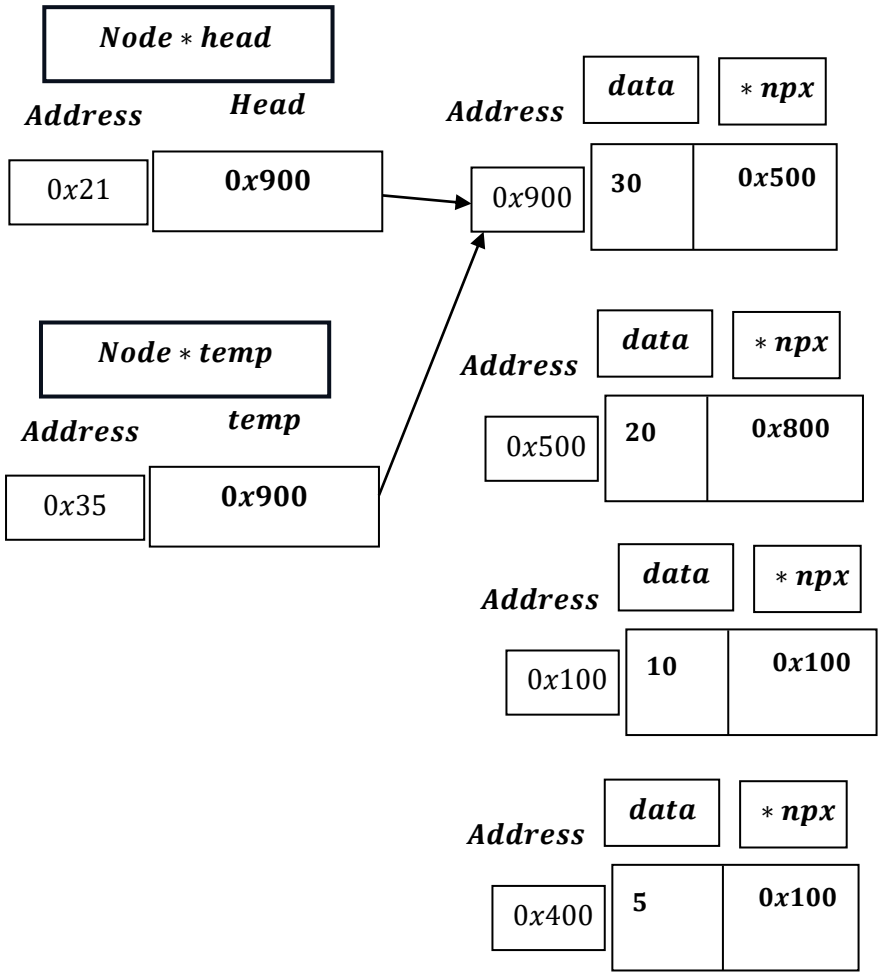


Then , it prints : "List is empty. Nothing to delete."

Return Void and Exit.

---

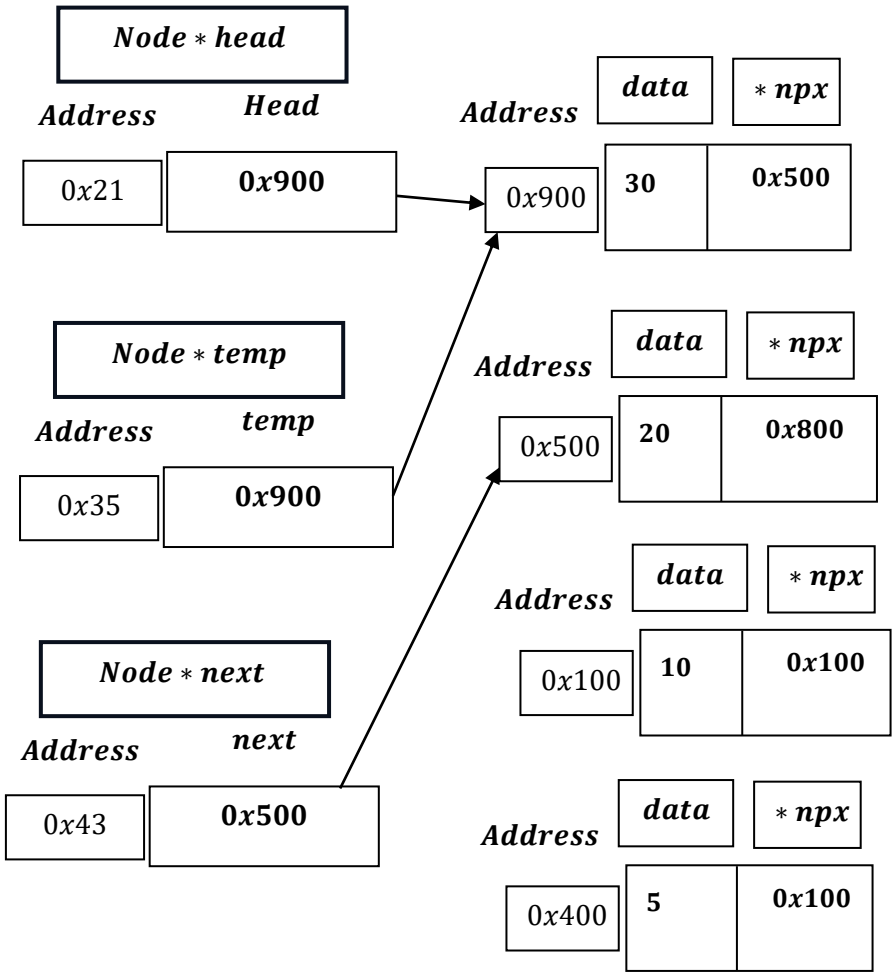
2. *When List Is Not Empty*




```
    Node *next = XOR(nullptr, head->npx);
```

$$Node * next = XOR(nullptr, head \rightarrow npx) \Longrightarrow 0x000 \oplus 0x500$$

$$= 0x500$$

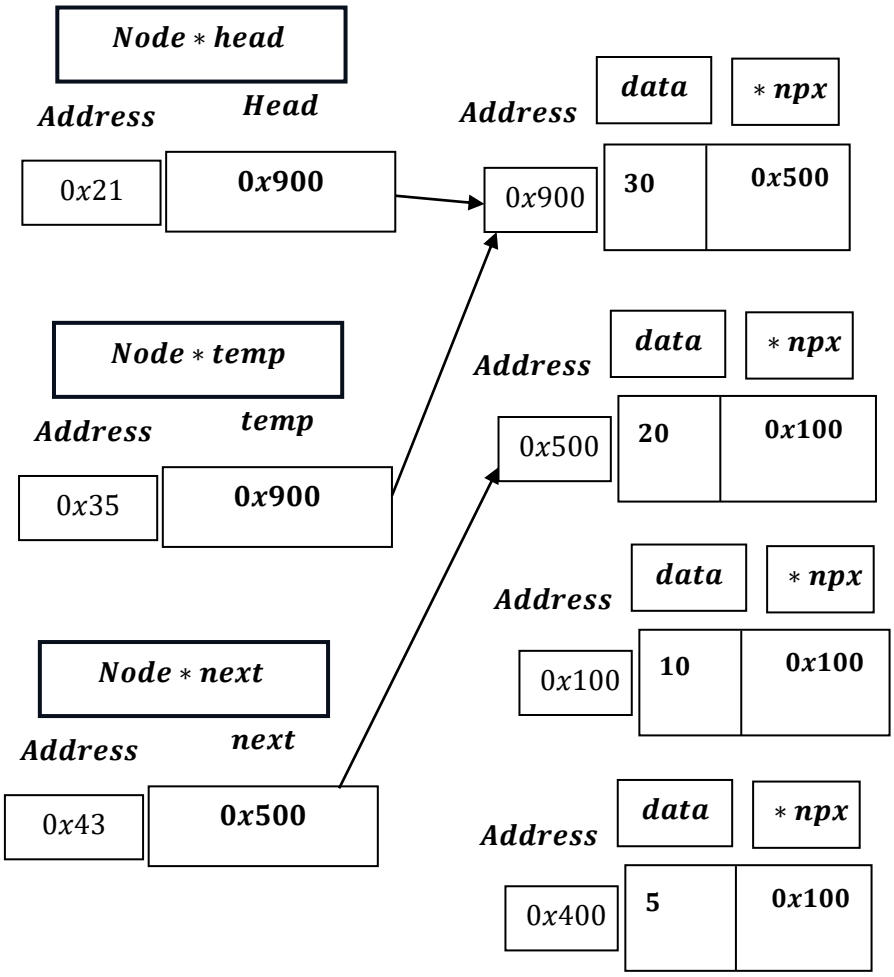


$\boldsymbol{Node * next = 0x500 \neq NULLPTR\ (0x000)}$.

```
        if (next != nullptr)
    {
        Node *nextNext = XOR(temp, next->npx);
        next->npx = XOR(nullptr, nextNext);
    }
```

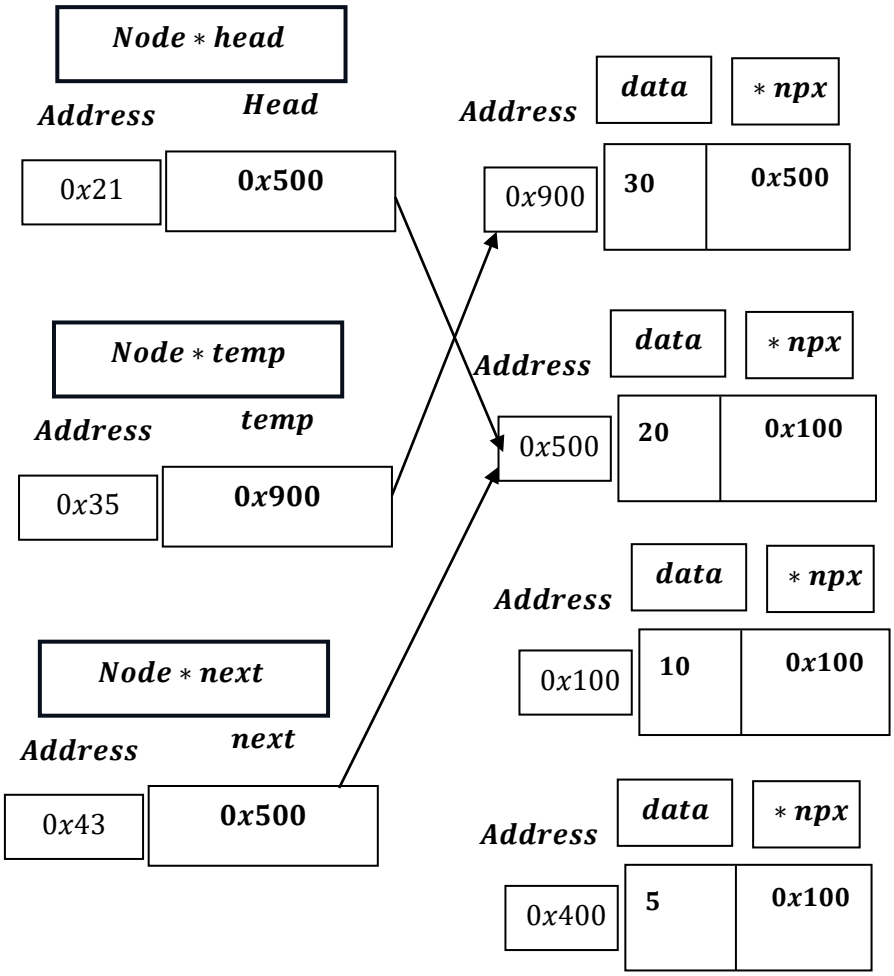
$$Node * nextNext = 0x900 \oplus 0x800 = 1001_2 \oplus 1000_2 = 0001_2 = 0x100$$

$$next \rightarrow npx = XOR\,(nullptr, nextNext) = 0x000 \oplus 0x100 = 0x100\,.$$




```
    head = next;
```

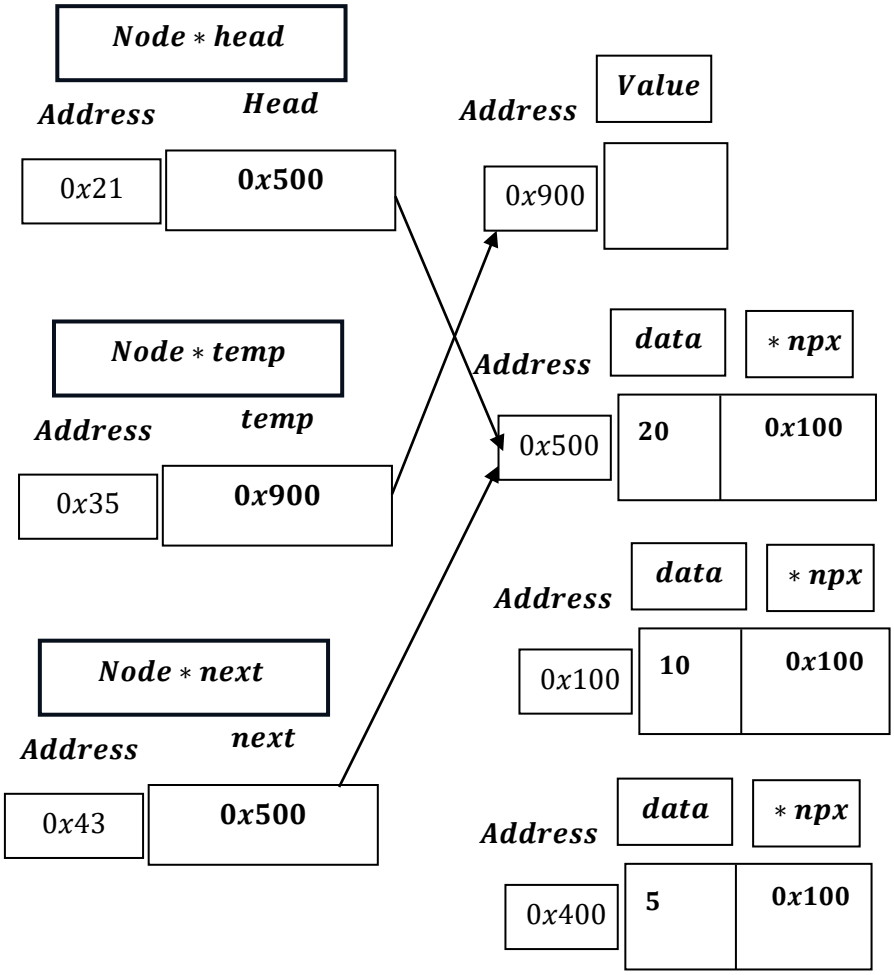
$$head = next = 0x500.$$



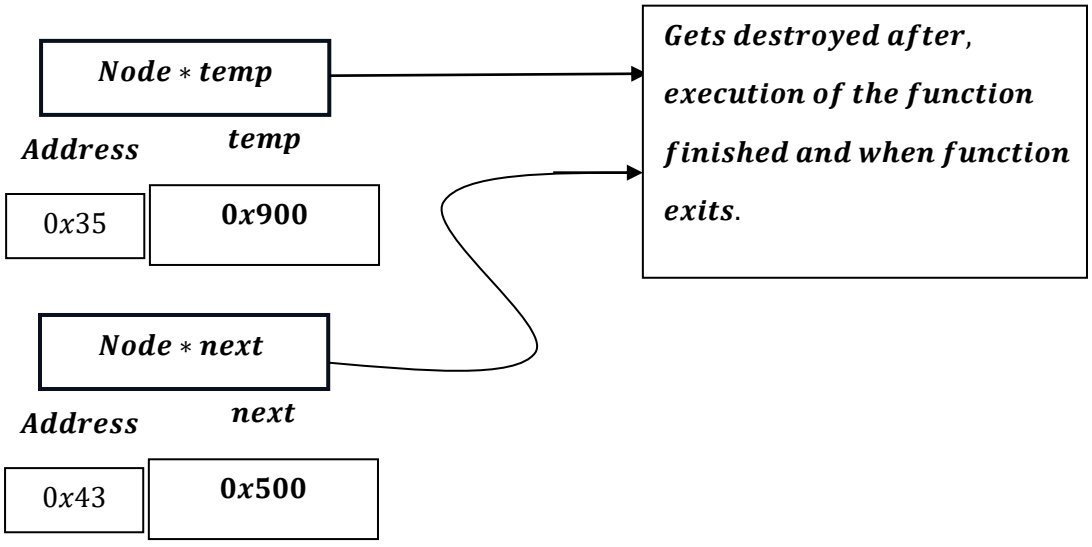

```
    cout << "Deleted " << temp->data << " from the
beginning." << endl;
```

Print: Deleted 30 from the beginning.

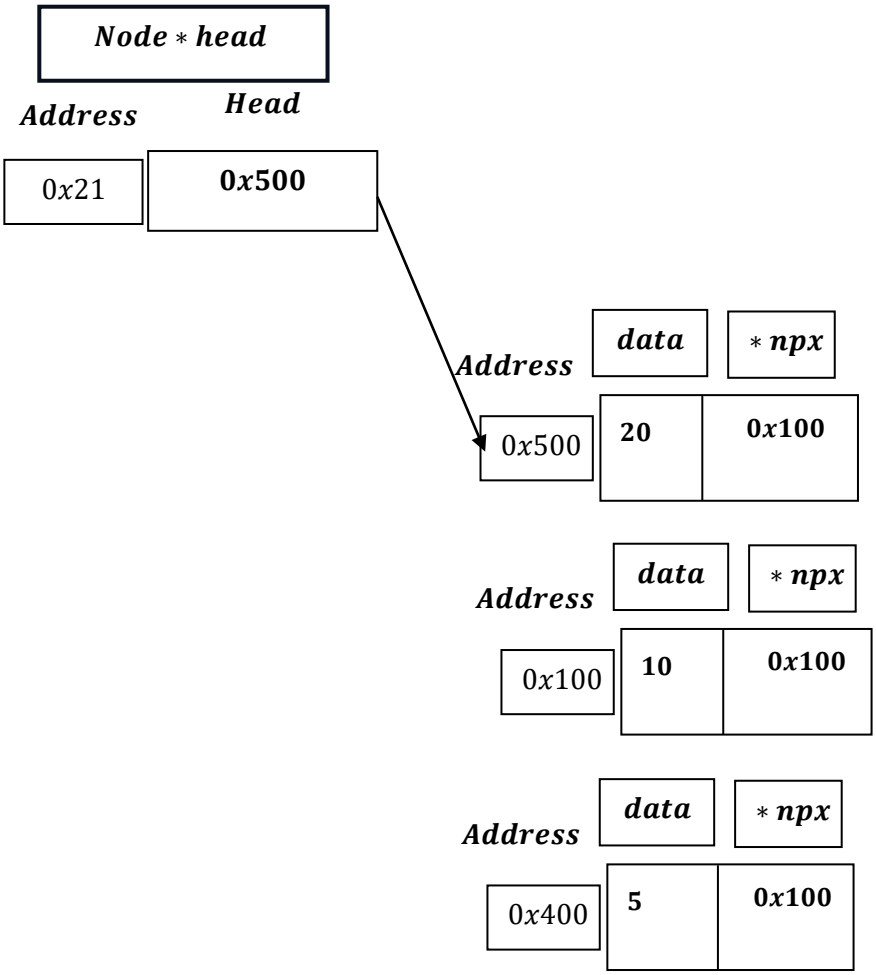
```
    free(temp);
```



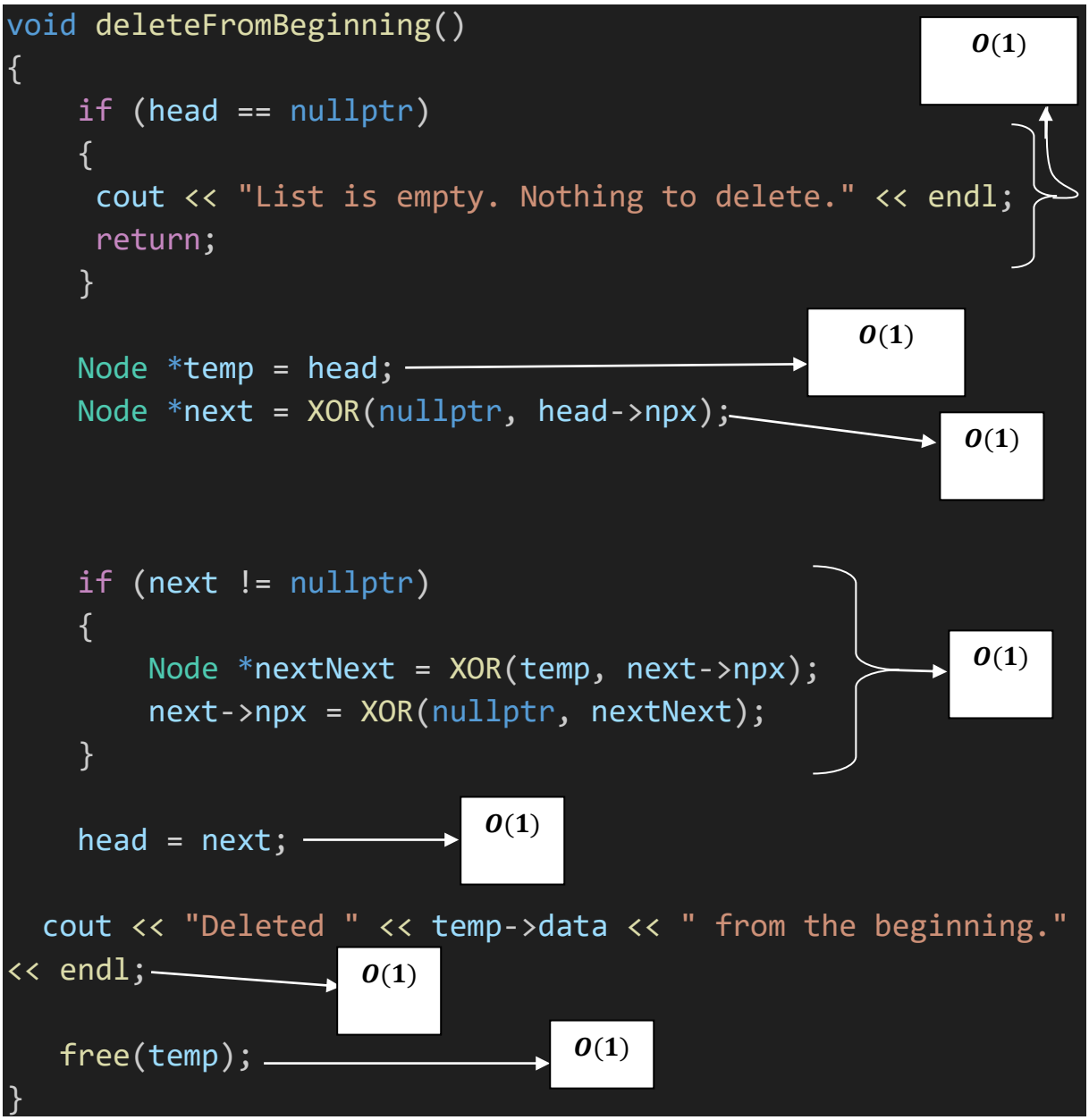
***As , functions ends*:**



And we are left with:



Time Complexity



$$\textbf{Best Case} = \Omega(1) = \textbf{Worst Case} = O(1) = \textbf{Average Case} = \Theta(1).$$

Memory Efficient Doubly Linked List	*Cases*	*Time Complexity*
Delete At Beginning	**1.** ***Best Case***	$\boldsymbol{\Omega(1)}$
	2. ***Worst Case***	$\boldsymbol{O(1)}$
	3. ***Average Case***	$\boldsymbol{\Theta(1)}$

****